

DATABASE SYSTEMS

DB Lecture 3

Database Design and Normalization

Topics covered in this lecture:

- Why database design matters: update, insertion, and deletion anomalies
- Functional dependencies and Armstrong's axioms
- First, Second, and Third Normal Form (1NF, 2NF, 3NF)
- Boyce-Codd Normal Form (BCNF)
- Mapping an ER diagram to relational tables
- Denormalization and trade-offs

Database Systems Lecture Series
Functional Dependencies, Anomalies, and Normal Forms

1. Why Database Design Matters

A poorly designed relational schema can store perfectly valid data yet still cause serious problems as the database is used and updated over time. **Normalization** is the systematic process of organizing the attributes and tables of a relational database to minimize data redundancy and eliminate a class of problems known as **update anomalies**.

1.1 A Motivating Example

Consider a single, un-normalized table that stores student enrollment data together with course and instructor information:

```
Enrollment(StudentID, StudentName, CourseID, CourseTitle,  
InstructorName, InstructorOffice, Grade)
```

Here, CourseTitle, InstructorName, and InstructorOffice all repeat for every student enrolled in the same course. This redundancy is not just wasteful of storage — it is actively dangerous, because it opens the door to three classes of anomalies.

1.2 Update Anomaly

If an instructor changes office, every row for every student enrolled in that instructor's courses must be updated. If even one row is missed, the database becomes inconsistent, showing two different offices for the same instructor.

1.3 Insertion Anomaly

A new course cannot be recorded until at least one student enrolls in it, because CourseID and CourseTitle only appear as part of an enrollment row. There is no way to insert information about a course on its own.

1.4 Deletion Anomaly

If the last student enrolled in a course drops it, and that row is deleted, information about the course itself (its title, instructor) is lost along with the enrollment record — even though the course still exists.

2. Functional Dependencies

The formal tool used to reason about redundancy and to drive normalization is the **functional dependency (FD)**. A functional dependency $X \rightarrow Y$ between two sets of attributes X and Y means: for any two tuples that agree on the values of X , they must also agree on the values of Y . Informally, X *determines* Y , or Y is *functionally dependent* on X .

2.1 Examples

- $\text{StudentID} \rightarrow \text{StudentName}$ — each student ID corresponds to exactly one name.
- $\text{CourseID} \rightarrow \text{CourseTitle}, \text{InstructorName}$ — each course has one title and one instructor.
- $\text{InstructorName} \rightarrow \text{InstructorOffice}$ — each instructor has one office.
- $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$ — a grade depends on the combination of student and course, not on either alone.

2.2 Armstrong's Axioms

Armstrong's axioms are a sound and complete set of inference rules for deriving all functional dependencies implied by a given set.

- **Reflexivity:** if Y is a subset of X , then $X \rightarrow Y$.
- **Augmentation:** if $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
- **Transitivity:** if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

From these three rules, several useful derived rules follow, including **union** ($X \rightarrow Y$ and $X \rightarrow Z$ implies $X \rightarrow YZ$) and **decomposition** ($X \rightarrow YZ$ implies $X \rightarrow Y$ and $X \rightarrow Z$).

2.3 Partial and Transitive Dependencies

A **partial dependency** exists when a non-key attribute depends on only part of a composite primary key, rather than the whole key. A **transitive dependency** exists when a non-key attribute depends on another non-key attribute, rather than directly on the key. Both are the specific targets eliminated by second and third normal form respectively, as we will see shortly.

3. First Normal Form (1NF)

A relation is in **First Normal Form** if every attribute holds only atomic (indivisible) values, and there are no repeating groups or multivalued attributes within a single row.

3.1 Violation Example

```
-- NOT in 1NF: PhoneNumbers holds multiple values in one cell
Student(StudentID, Name, PhoneNumbers)
1 | Ayesha | '0301-1111111, 0333-2222222'
```

3.2 1NF-Compliant Redesign

```
Student(StudentID, Name)
StudentPhone(StudentID, PhoneNumber)
-- Each phone number now occupies its own row
```

1NF is considered part of the very definition of the relational model; a table that violates 1NF is not, strictly speaking, a valid relation at all.

4. Second Normal Form (2NF)

A relation is in **Second Normal Form** if it is in 1NF and every non-key attribute is **fully functionally dependent** on the entire primary key — that is, there are no partial dependencies. 2NF is only a meaningful concept for relations with a composite primary key; any relation with a single-attribute primary key is automatically in 2NF once it is in 1NF.

4.1 Violation Example

```
Enrollment(StudentID, CourseID, StudentName, CourseTitle, Grade)
Primary Key: {StudentID, CourseID}
```

```
-- StudentName depends only on StudentID (partial dependency)
-- CourseTitle depends only on CourseID (partial dependency)
-- Grade depends on the full key {StudentID, CourseID} -- OK
```

4.2 2NF-Compliant Decomposition

```
Student(StudentID, StudentName)
Course(CourseID, CourseTitle)
Enrollment(StudentID, CourseID, Grade)
```

By moving StudentName and CourseTitle into their own relations, each keyed on the attribute they truly depend on, we eliminate the partial dependencies and the associated update anomalies.

5. Third Normal Form (3NF)

A relation is in **Third Normal Form** if it is in 2NF and no non-key attribute is **transitively dependent** on the primary key — that is, every non-key attribute depends directly on the key, and on nothing but the key.

5.1 Violation Example

```
Course(CourseID, InstructorName, InstructorOffice)
Primary Key: CourseID

-- CourseID -> InstructorName (direct)
-- InstructorName -> InstructorOffice (direct)
-- Therefore CourseID -> InstructorOffice (transitive, via InstructorName)
```

5.2 3NF-Compliant Decomposition

```
Course(CourseID, InstructorName)
Instructor(InstructorName, InstructorOffice)
```

This removes the transitive dependency: InstructorOffice is now stored once per instructor, not once per course. A common mnemonic for 3NF is: "every non-key attribute must depend on the key, the whole key, and nothing but the key."

6. Boyce-Codd Normal Form (BCNF)

BCNF is a slightly stronger version of 3NF. A relation is in BCNF if, for every nontrivial functional dependency $X \rightarrow Y$, X is a **super key** of the relation. This closes a subtle gap that 3NF leaves open: 3NF allows certain anomalies to persist when a relation has multiple overlapping candidate keys.

6.1 Example Where 3NF Is Satisfied but BCNF Is Not

Consider a relation `StudentAdvisor(StudentID, Major, Advisor)`, where each student can have multiple majors, each major is advised by several advisors, but each advisor advises students in only one major:

```
StudentAdvisor(StudentID, Major, Advisor)
FDs: {StudentID, Major} -> Advisor (candidate key)
Advisor -> Major (Advisor is not a super key!)
```

This relation is in 3NF (Major is a prime attribute since it participates in a candidate key, so the FD $\text{Advisor} \rightarrow \text{Major}$ does not violate 3NF's specific rule), but it is not in BCNF, because `Advisor` determines `Major` without `Advisor` being a super key. The fix is to decompose further:

```
AdvisorMajor(Advisor, Major)
StudentAdvisor(StudentID, Advisor)
```

6.1b Testing Whether a Decomposition Is Lossless

Whenever a relation is decomposed into two smaller relations `R1` and `R2`, it is essential to verify the decomposition is **lossless (non-additive)**, meaning the original relation can always be reconstructed exactly by joining `R1` and `R2` back together, with no spurious rows introduced. A sufficient condition is that the common attributes of `R1` and `R2` form a candidate key of at least one of the two relations. All of the decompositions performed in this lecture satisfy this condition, since we always split off a set of attributes together with the key attribute they depend on.

6.2 BCNF vs. 3NF Trade-off

BCNF eliminates strictly more redundancy than 3NF, but it is not always possible to achieve a BCNF decomposition that also preserves all the original functional dependencies without recombination (i.e., without needing a join to check them). 3NF, by contrast, always allows a dependency-preserving decomposition. In practice, most real-world schemas that reach 3NF are already in BCNF, so the distinction mainly matters for relations with unusual, overlapping candidate key structures.

7. Higher Normal Forms (Brief Overview)

Beyond BCNF, additional normal forms address more subtle types of redundancy that arise from multivalued and join dependencies, though they are used less frequently in practice.

- **Fourth Normal Form (4NF):** eliminates redundancy caused by **multivalued dependencies** — independent one-to-many relationships stored in the same table (e.g. a table combining a student's skills and a student's hobbies leads to a combinatorial explosion of rows).
- **Fifth Normal Form (5NF):** eliminates redundancy caused by **join dependencies** that are not implied by candidate keys, ensuring a relation cannot be losslessly decomposed any further without loss of meaning.

In most real-world database courses and industry practice, achieving 3NF or BCNF is considered sufficient; 4NF and 5NF are primarily of theoretical and specialized interest.

7.1 Summary Table of Normal Forms

Normal Form	Eliminates
1NF	Repeating groups / multivalued cells.
2NF	Partial dependencies on a composite key.
3NF	Transitive dependencies through non-key attributes.
BCNF	Dependencies where the determinant is not a super key.
4NF	Redundancy from independent multivalued dependencies.
5NF	Redundancy from join dependencies not implied by keys.

8. Mapping an ER Diagram to Relational Tables

Database design typically starts with an ER diagram (DB Lecture 1) and is then systematically converted into a set of relational tables, which are subsequently checked against the normal forms above.

8.1 Mapping Rules

- 1 **Regular entity type** → a table with the entity's simple attributes as columns and its key attribute as the primary key.
- 2 **Weak entity type** → a table whose primary key is a composite of its own partial key plus the primary key of its owner (identifying) entity.
- 3 **1:1 relationship** → add the primary key of one entity as a foreign key in the table of the other (commonly the entity with total participation), or merge into a single table if appropriate.
- 4 **1:N relationship** → add the primary key of the 'one' side as a foreign key in the table on the 'many' side.
- 5 **M:N relationship** → create a new, separate table (sometimes called a bridge or junction table) whose primary key is the combination of the primary keys of both participating entities, plus any attributes of the relationship itself.
- 6 **Multivalued attribute** → create a new table with a foreign key back to the owning entity, one row per value.

8.2 Worked Example

Recall the STUDENT – ENROLLS_IN (M:N, with attribute Grade) – COURSE example from DB Lecture 1. Applying rule 5 above yields exactly the three-table design we arrived at through normalization in Section 4 of this lecture:

```
Student(StudentID, StudentName, ...)
Course(CourseID, CourseTitle, ...)
Enrollment(StudentID, CourseID, Grade)
FOREIGN KEY (StudentID) REFERENCES Student
FOREIGN KEY (CourseID) REFERENCES Course
```

This is a reassuring convergence: careful ER-to-relational mapping and bottom-up normalization tend to arrive at the same well-structured design.

8.3 Why Both Approaches Are Taught Together

In practice, database designers use both approaches together. The ER-to-relational mapping gives a fast, intuitive starting schema directly from the conceptual design produced with stakeholders. Normalization is then used as a rigorous **verification** step, checking the resulting tables against the functional dependencies implied by the business rules to confirm no anomaly-causing redundancy has crept in, and to catch design mistakes such as attributes placed on the wrong entity, or relationships that were not modeled with enough granularity in the original ER diagram.

9. Denormalization

Denormalization is the deliberate process of introducing some redundancy back into a normalized schema, typically to improve read performance for specific, well-known query patterns at the cost of increased storage and more complex update logic.

9.1 When Denormalization Might Be Appropriate

- Reporting and analytics (data warehouse) systems, where reads vastly outnumber writes and join-heavy queries are expensive.
- Applications with strict, known low-latency read requirements, where an extra join at query time is unacceptable.
- Storing a computed/derived value (e.g. OrderTotal) to avoid recomputing it from detail rows on every read, provided a clear strategy exists to keep it in sync.

9.1b Common Denormalization Techniques

- **Adding redundant columns:** copying a frequently-needed attribute from a related table directly into the table being queried, avoiding a join.
- **Pre-joined / summary tables:** materializing the result of an expensive join or aggregation into its own table, refreshed periodically or via triggers.
- **Storing derived/calculated values:** keeping a running total or count as a stored column, updated whenever the underlying detail rows change.
- **Combining tables:** merging two tables that are almost always queried together into one, trading some redundancy for fewer joins.

9.2 Risks

- Reintroduces the very update, insertion, and deletion anomalies normalization was designed to eliminate.
- Requires additional application logic or triggers to keep redundant copies of data synchronized.
- Should generally be applied selectively and deliberately, after measuring an actual performance need, rather than as a default design choice.

9b. A Complete Worked Example

To consolidate the normalization process, let us fully normalize a realistic unnormalized relation step by step, from 1NF through 3NF.

9b.1 The Starting Point

```
Order(OrderID, OrderDate, CustomerID, CustomerName, CustomerCity,
ProductList)
-- ProductList = '(P1,Widget,2,9.99); (P2,Gadget,1,19.99)' -- NOT 1NF
```

9b.2 Step 1 — Convert to 1NF

We split the repeating ProductList into individual rows, one per ordered product, so that each cell holds a single atomic value.

```
Order(OrderID, OrderDate, CustomerID, CustomerName, CustomerCity,
ProductID, ProductName, Quantity, UnitPrice)
Primary Key: {OrderID, ProductID}
```

9b.3 Step 2 — Convert to 2NF

The primary key is composite: {OrderID, ProductID}. We check each non-key attribute: OrderDate, CustomerID, CustomerName, and CustomerCity depend only on OrderID (a partial dependency); ProductName and UnitPrice depend only on ProductID (also partial); only Quantity genuinely depends on the whole key. We remove the partial dependencies by splitting the relation:

```
Order(OrderID, OrderDate, CustomerID, CustomerName, CustomerCity)
Product(ProductID, ProductName, UnitPrice)
OrderLine(OrderID, ProductID, Quantity)
```

9b.4 Step 3 — Convert to 3NF

Within the Order relation, CustomerName and CustomerCity depend on CustomerID, not directly on OrderID — a transitive dependency ($\text{OrderID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerName, CustomerCity}$). We remove it by splitting out Customer:

```
Order(OrderID, OrderDate, CustomerID)
Customer(CustomerID, CustomerName, CustomerCity)
Product(ProductID, ProductName, UnitPrice)
OrderLine(OrderID, ProductID, Quantity)
```

This four-table design is in 3NF (and, since there is no relation with overlapping candidate keys, also in BCNF): every non-key attribute depends on the whole key of its own relation and nothing else, redundancy has been eliminated, and all four of the update, insertion, and deletion anomalies from Section 1 are resolved.

10. Review Questions

- 1 Define update, insertion, and deletion anomalies, and give an original example of each in a single un-normalized table.
- 2 What is a functional dependency? Give an example of a partial dependency and a transitive dependency.
- 3 State the requirement for 1NF, 2NF, and 3NF in your own words.
- 4 Explain why BCNF is considered stronger than 3NF, using the StudentAdvisor example or one of your own.
- 5 Given the unnormalized relation Invoice(InvoiceID, CustomerID, CustomerName, ProductID, ProductName, Quantity, UnitPrice), identify the functional dependencies and normalize it to 3NF, showing each intermediate step.
- 6 Describe the mapping rule for converting an M:N relationship in an ER diagram into relational tables.
- 7 Under what circumstances might a database designer deliberately denormalize a schema? What risks does this introduce?

11. Summary

This lecture examined why raw, unstructured tables cause update, insertion, and deletion anomalies, and how the formal tool of the functional dependency lets us reason precisely about these problems. We walked through 1NF, 2NF, 3NF, and BCNF, each removing a progressively subtler form of redundancy, and connected this bottom-up normalization process with the top-down approach of mapping an ER diagram directly into relational tables. Finally, we discussed denormalization as a deliberate, measured trade-off of redundancy for performance.

Key Takeaways

- Normalization exists to eliminate redundancy and the update/insertion/deletion anomalies it causes.
- Each normal form (1NF, 2NF, 3NF, BCNF) targets a specific category of dependency: repeating groups, partial dependencies, transitive dependencies, and non-super-key determinants, respectively.
- In practice, most schemas are designed by mapping an ER diagram to tables and then verifying the result satisfies at least 3NF.
- Denormalization is a deliberate, measured exception to normalization, applied for performance reasons, not a substitute for good design.

Looking Ahead

DB Lecture 4 turns to what happens once a well-designed database is actually being used concurrently by many users: **transactions**, the ACID properties, concurrency control, and indexing for efficient query performance.

11b. Common Normalization Mistakes to Avoid

- **Over-normalizing:** splitting tables so finely that even simple queries require many joins, hurting both performance and readability without a corresponding integrity benefit.
- **Confusing 2NF with 3NF:** remember 2NF is only relevant for composite keys (partial dependency), while 3NF applies regardless of key shape (transitive dependency).
- **Forgetting to check BCNF** when a relation has more than one candidate key that overlaps in attributes — 3NF alone may not be enough in that specific situation.
- **Normalizing without considering query patterns:** normalization is about correctness and redundancy, not performance; some carefully chosen denormalization may still be needed afterward, as discussed in Section 9.
- **Skipping the lossless-join check** when decomposing a relation, which can, in rare cases, introduce spurious tuples when tables are joined back together.

Keeping these pitfalls in mind will help you apply normalization as a practical design discipline rather than a purely mechanical exercise.

12. Glossary of Key Terms

Term	Definition
Normalization	The process of organizing tables to minimize redundancy and eliminate update anomalies.
Functional dependency	A constraint $X \rightarrow Y$ meaning X's value determines Y's value.
Partial dependency	A non-key attribute depending on only part of a composite key.
Transitive dependency	A non-key attribute depending on another non-key attribute rather than the key directly.
1NF	Every attribute holds a single, atomic value.
2NF	1NF, with no partial dependencies on the primary key.
3NF	2NF, with no transitive dependencies on the primary key.
BCNF	Every determinant of a nontrivial FD is a super key.
Denormalization	Deliberately reintroducing redundancy to improve read performance.

Keep this glossary handy while working through the review questions in Section 10, and revisit it before attempting the worked example in Section 9b if any term feels unclear.